

华东师范大学数据学院上机实践报告

课程名称： 操作系统

年级： 2022 级

上机实践成绩：

指导教师： 翁楚良

姓名：

上机实践名称： 实验一 xv6-lab-util 学号：

上机实践日期： 2024.3.16

一、实验目的

熟悉如何在 xv6 环境下写代码

二、实验内容

完成 sleep、pingpong、primes、find、xargs 五个作业

三、使用环境

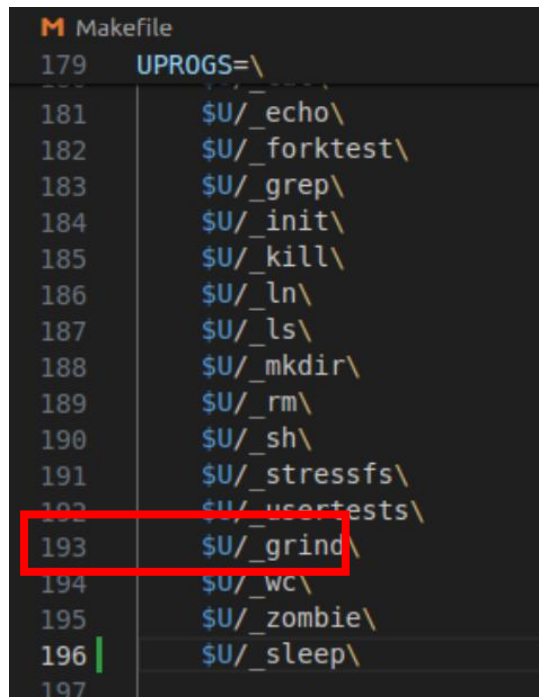
Ubuntu 64-VMware Workstation

四、实验过程及结果

Task1: sleep

作业一要求利用系统调用函数 sleep 完成 sleep n，熟悉 lab 操作流程。

首先创建 user/sleep.c，在 Makefile 中的 UPROGS 下添加\$U/_sleep\，如下：



```
M Makefile
179 UPROGS=\
180     $U/_echo\
181     $U/_forktest\
182     $U/_grep\
183     $U/_init\
184     $U/_kill\
185     $U/_ln\
186     $U/_ls\
187     $U/_mkdir\
188     $U/_rm\
189     $U/_sh\
190     $U/_stressfs\
191     $U/_usertests\
192     $U/_grind\
193     $U/_wc\
194     $U/_zombie\
195     $U/_sleep\
196
197
```

编写 sleep.c:

user.h 为 xv6 提供系统函数，types.h 为其提供变量类型，stat.h 是声明文件或目录属性数据结构。若忘记传参，程序需要报错。atoi(user/ulib.c) 可以将默认传参的 string 转换为 int。

```
user > C sleep.c
1  #include "kernel/types.h"
2  #include "kernel/stat.h"
3  #include "user/user.h"
4
5  int main(int argc, char *argv[])
6  {
7      if(argc < 2)
8      {
9          fprintf(2, "Usage: sleep number\n");
10         exit(1);
11     }
12     int time = atoi(argv[1]);
13     sleep(time);
14     exit(0);
15 }
```

开启 qemu：输入命令 make qemu，测试输入 sleep 10 发现停顿了几秒，

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ sleep 10
$
```

按 ctrl-a x 退出 qemu，输入 ./grade-lab-util sleep 测试。

```
make: "kernel/kernel"已是最新。
== Test sleep, no arguments == sleep, no arguments: OK (3.1s)
== Test sleep, returns == sleep, returns: OK (2.5s)
== Test sleep, makes syscall == sleep, makes syscall: OK (2.6s)
```

如上，测试成功。

Task2: pingpong

作业二要求利用管道实现进程间的通信：父进程发送 ping，子进程收到后发送 pong，父进程收到后将其打印出来。

我们需要利用 pipe()函数创建管道，接受一个长度为 2 的数组，数组下标 0 为读端，1 为写端，并利用 fork()创建新的进程，利用 read、write 函数读写管道。

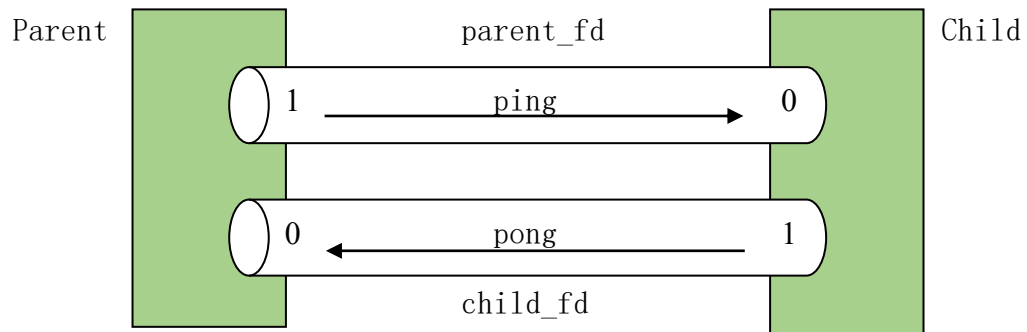
```
user > C pingpong.c
1  #include "kernel/types.h"
2  #include "kernel/stat.h"
3  #include "user/user.h"
4
5  int
6  main(int argc, char *argv[])
7  {
8      int pid;
9      int parent_fd[2]; //parent to child
10     int child_fd[2];  //child to parent
11
12     //create pipe
13     pipe(child_fd);
14     pipe(parent_fd);
15
16     pid = fork(); //fork
17     //Child Progress
18     if(pid == 0)
19     {
20         close(parent_fd[1]);
21         close(child_fd[0]);
22
23         char buf[16];
24         //child receive ping, print
25         if(read(parent_fd[0], buf, sizeof(buf)) == 1)
26         {
27             printf("%d: received ping\n", pid);
28         }
29         //child to parent
30         write(child_fd[1], "pong", 1);
31         exit(0);
32     }
33
34     //Parent Progress
35     else
36     {
37         close(parent_fd[0]);
38         close(child_fd[1]);
39         write(parent_fd[1], "ping", 1);
40         char buf[16];
41         if(read(child_fd[0], buf, sizeof(buf)) == 1)
42         {
43             printf("%d: received pong\n", pid);
44         }
45     }
46     exit(0);
47 }
```

read() 用于从文件描述符 fd 中读取数据，并将数据存储在缓冲区中。write() 用于

将数据从缓冲区写入到文件描述符中。

由于管道是单向的，我们需要在创建子进程前设置两个双向管道。然后通过 `fork()` 创建子进程，`fork()` 返回两种不同值，父进程得到子进程 `pid`，子进程得到返回值 0。

绘制图可以更好理解管道是如何实现父进程与子进程之间的通信的：



父进程向子进程发出 ping，需要关闭 `parent_fd[0]` 即父进程的读端，写 ping，而子进程要关闭 `child_fd[1]` 即子进程的写端，并通过 `parent_fd[0]` 即管道的读端去读并打印出来；子进程向父进程回复 pong，需要关闭 `child_fd[0]` 即子进程的读端，写 pong，而父进程要关闭 `child_fd[1]` 即父进程的写端，并通过 `child_fd[0]` 即管道的读端去读并打印出来。

测试结果：

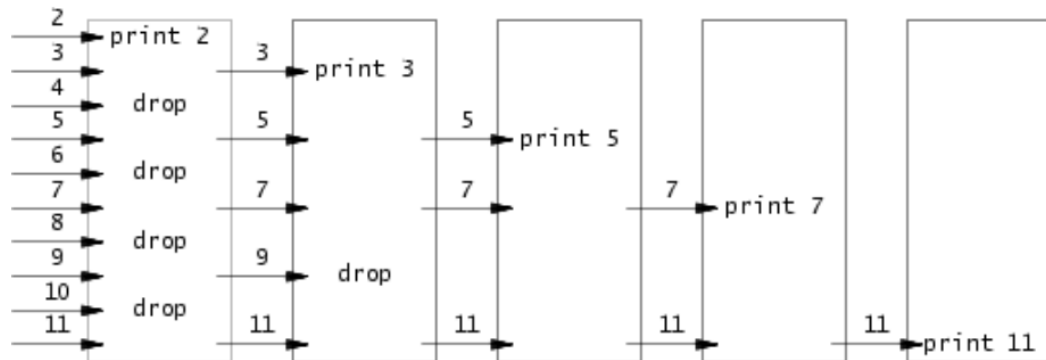
```
$ pingpong
0: received ping
17: received pong
```

```
make: "kernel/kernel"已是最新。
== Test pingpong == pingpong: OK (5.7s)
```

Task3: primes

作业三要求用 pipe 编写素数筛的并发版本，需要用 fork 和 pipe 实现输入为 $2 \sim 35$ ，输出为其之间的所有素数。此实验基于 Unix pipe 的发明者 Doug McIlroy 提出的多进程素数筛案例。

如下图所示，第一次将 $2 \sim 35$ 给到一个进程，此进程发现第一个数为 2，则输出 2，并将不能被 2 除尽的数发送到下一个进程；下一个进程发现第一个数为 3，则输出 3，并将不能被 3 除尽的数发送到下一个进程……以此类推。



Hints 中提到，要小心关闭不需要的文件描述符，但同时第一个进程需要等待所有进程的结束。因此不能不等待直接退出，每个父进程使用 `wait(0)` 来等待子进程完成。

`primes.c` 中创建 `child()` 函数来反复调用它，关闭 `parent_fd[1]` 即子进程的写端，如果 `parent_fd[0]` 已经读不到数字 `i` 了，则说明全部素数已经输出完毕，所以关闭子进程管道直接退出。创建子进程到孙进程的管道 `child_fd`，孙进程则递归调用 `child()` 函数。子进程到孙进程，子进程不需要读，通过 `num % i` 取模来判断，不能整除则发到下个进程，关闭子进程写端，并等待所有其子进程结束后结束。

`main` 函数中创建父进程到子进程的管道，子进程调用 `child()` 函数，父进程则关闭 `parent_fd[0]` 即子进程的读端，循环将 $2 \sim 35$ 的数传到子进程，传的是 `i` 的地址，随后关闭 `parent_fd[1]`，并等待子进程结束后父进程结束。

具体代码如下：

```
user > C primes.c
1  #include "kernel/types.h"
2  #include "kernel/stat.h"
3  #include "user/user.h"
4
5  void child(int parent_fd[])
6  {
7      close(parent_fd[1]);
8      int i;
9      if(read(parent_fd[0], &i, sizeof(i)) == 0)
10     {
11         //no numbers left, close pipe and exit
12         close(parent_fd[0]);
13         exit(0);
14     }
15     printf("prime %d\n", i);
16     int num = 0;
17     //child to grandchild
18     int child_fd[2];
19     pipe(child_fd);
20     int pid;
21     //grandchild progress
22     if((pid = fork()) == 0)
23     {
24         child(child_fd);
25     }
26     //child progress
27     else
28     {
29         close(child_fd[0]);
30         while(read(parent_fd[0], &num, sizeof(num)) > 0)
31         {
32             //write condition
33             if(num % i != 0)
34             {
35                 write(child_fd[1], &num, sizeof(num));
36             }
37         }
38         close(child_fd[1]);
39         wait(0);
40     }
41     exit(0);
42 }
43
44 int main(int argc, char *argv[])
45 {
46     int parent_fd[2];
47     pipe(parent_fd);
48     int pid;
49     //child progress
50     if((pid = fork()) == 0)
51     {
52         child(parent_fd);
53     }
54     //parent progress
55     else
56     {
57         close(parent_fd[0]);
58         for(int i = 2; i <= 35; i++)
59         {
60             write(parent_fd[1], &i, sizeof(i));
61         }
62         close(parent_fd[1]);
63         //wait for child progress
64         wait(0);
65     }
66     //parent progress end
67     exit(0);
68 }
```

测试结果:

```
$ primes
prime 2
prime 3
prime 5
prime 7
prime 11
prime 13
prime 17
prime 19
prime 23
prime 29
prime 31
```

```
make: "kernel/kernel"已是最新。
== Test primes == primes: OK (3.7s)
```

Task4: find

作业四要求编写一个简单版本的 UNIX 查找程序：查找目录中具有特定名称的所有文件。Hints 提醒查看 `user/ls.h` 了解如何读取目录，用递归方法处理子文件，但除去“.”和“..”。

代码内涉及到结构体，查看 `/kernel/stat.h`，`stat` 定义文件本身的信息。

```
kernel > C stat.h
1  #define T_DIR      1  // Directory
2  #define T_FILE     2  // File
3  #define T_DEVICE   3  // Device
4
5  struct stat {
6      int dev;        // File system's disk device
7      uint ino;       // Inode number
8      short type;     // Type of file
9      short nlink;    // Number of links to file
10     uint64 size;     // Size of file in bytes
11 };
```

`/kernel/fs.h` 中 `dirent` 定义文件名和文件名长。`DIRSIZ` 表示最大的文件名长度。


```
// Directory is a file containing a sequence of dirent structures.
#define DIRSIZ 14

struct dirent {
    ushort inum;
    char name[DIRSIZ];
};
```

```
user > C find.c
1  #include "kernel/types.h"
2  #include "kernel/stat.h"
3  #include "user/user.h"
4  #include "kernel/fs.h"
5  #include "kernel/fcntl.h"
6
7  #define stdin 0
8  #define stdout 1
9  #define stderr 2
10
11 char* fmtname(char* path)
12 {
13     char* p;
14     for(p = path + strlen(path); p >= path && *p != '/'; p--)
15         ;
16     return p + 1;
17 }
18
19 void find(char* path, const char* filename)
20 {
21     char buf[512], *p;
22     int fd;
23     struct dirent de;
24     struct stat st;
25
26     if((fd = open(path, O_RDONLY)) < 0) //open path file descriptors
27     {
28         fprintf(stderr, "find: cannot open %s\n", path);
29         return;
30     }
31     if(fstat(fd, &st) < 0)
32     {
33         fprintf(stderr, "find: cannot stat %s\n", path);
34         close(fd);
35         return;
36     }
37
38     switch(st.type)
39     {
40     case T_FILE: //FILE type, print directly
41         if(strcmp(filename, fmtname(path)) == 0)
42         {
43             printf("%s\n", path);
44         }
45         break;
```

将 ls.c 中可以重复使用的代码移植到 find.c 中，包括 ls 函数的结构部分和 fmtname() 函数。其实现与 ls.c 中基本相同，而在 T_DIR 中，不需要输出所有文件的所有信息，而是进行筛选符合条件的名字再输出 buf 缓冲区中存储的相对路径即可。


```

46     case T_DIR:           //DIR type, size too long?
47         if(strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf))
48         {
49             printf("find: path too long\n");
50             return;
51         }
52         strcpy(buf, path); //copy path to buf
53         p = buf + strlen(buf); //move pointer
54         *p++ = '/'; //equal to *(p++)
55
56         while(read(fd, &de, sizeof(de)) == sizeof(de))
57         {
58             if(de.inum == 0 || strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0)
59             {
60                 continue;
61             }
62             memmove(p, de.name, DIRSIZ);
63             p[DIRSIZ] = 0;
64             if(stat(buf, &st) < 0)
65             {
66                 printf("find: cannot stat %s\n", buf);
67                 continue;
68             }
69             find(buf, filename);
70         }
71         break;
72     }
73     close(fd);
74 }
75
76 int main(int argc, char* argv[])
77 {
78     if(argc < 3)
79     {
80         printf("usage: find path filename\n");
81     }
82     for(int i = 2; i < argc; i++)
83     {
84         find(argv[1], argv[i]);
85     }
86     exit(0);
87 }

```

find.c 与 ls.c 读取目录和文件信息的方式非常类似，定义 buf、p 指针和 open 路径后打开的文件描述符 fd，还有描述文件名称和大小的 dirent、描述文件信息的 stat。如果打开路径表示 <0，则说明路径不对，打不开。并且 fstat() 函数返回的数组具有该文件的统计信息，若 <0，说明没有文件的具体信息。

通过 switch() 判断不同的文件类型 st.type：如果是文件 FILE，则直接输出路径；如果是目录 DIR，则判断是否太长，文件名长度+路径长度>buf 长度，则报错。之后用 strcpy(buf, path) 将 path 读到 buf (e.g. './a/b')，然后再往后取 strlen 长度个地址，*p++ = '/' ('./a/b/')。

使用 de.name 对当前目录排除 "." 和 ".." 后进行递归遍历，同时对路径名进行匹配，匹配则输出。

通过查阅资料知，open() 函数对文件设备打开操作，会返回一个句柄 fd，通过此句柄对设备文件进行读写操作。int open(const char* pathname, int flags)，pathname 表示路径地址，flags 用来控制打开文件的模式，如 O_RDONLY 以只读方式打开文件，O_WRONLY 以只写方式打开文件，O_RDWR 以可读写方式打开文件。

`void *memmove(void *str1, const void *str2, size_t n)`从 `str2` 复制到 `str1`，在重叠内存块方面，`memmove()` 比 `memcpy()` 更安全，如果目标区域和源区域有重叠，`memmove()` 能够保证源串在被覆盖之前将重叠区域的字节拷贝到目标区域中，复制后源区域的内容会被更改。参数 `str1` 指向用于存储复制内容的目标数组，类型强制转换为 `void*` 指针，`str2` 指向要复制的数据源，类型强制转换为 `void*` 指针，`n` 是要被复制的字节数。

测试结果：

```
make: "kernel/kernel"已是最新。
== Test find, in current directory == find, in current directory: OK (4.4s)
== Test find, recursive == find, recursive: OK (4.2s)
```

Task5: xargs

作业五要求编写 `xargs` 程序，从标准输入读取行为并为每行运行一个命令，将该行作为参数提供给命令。它的主要思想是提取前一个作为参数的命令结果，然后构建一个子进程将读取的参数传给 `xargs` 指定的命令运行，使用 `wait` 等待子进程结束。

将前一个命令的输出附加到后一个命令上 `echo hello too | xargs echo bye`，则 `echo hello too` 的输出 `hello too`，作为附加参数给 `echo bye` 指令。

Hints 提示用 `fork` 和 `exec` 在输入的每一行上调用命令，在父进程中使用 `wait` 等待子进程完成命令。在 `kernel/param.h` 中声明了 `MAXARG`。

```
user > C xargs.c
1  #include "kernel/types.h"
2  #include "kernel/stat.h"
3  #include "user/user.h"
4  #include "kernel/param.h"
5  #include "kernel/fs.h"
6
7  #define stdin 0
8  #define stdout 1
9  #define stderr 2
10 #define MAX_ARG_LEN 1024
11
12 int main(int argc, char* argv[])
13 {
14     int pid, n, buf_index = 0;
15     char buf, arg[MAX_ARG_LEN], *args[MAXARG];
16
17     if(argc < 2)
18     {
19         fprintf(stderr, "usage: xargs <command> ...\n");
20         exit(0);
21     }
22     for(int i = 1; i < argc; ++i)
23     {
24         args[i-1] = argv[i];
25     }
26
27     while((n = read(stdin, &buf, 1)) > 0)
28     {
29         if(buf == '\n')
30         {
31             arg[buf_index] = 0;
32             if((pid = fork()) < 0)
33             {
34                 fprintf(stderr, "fork error...\n");
35                 exit(0);
36             }
37             else if(pid == 0)
38             {
39                 args[argc - 1] = arg;
40                 args[argc] = 0;
41                 exec(args[0], args);
42             }
43             else
44             {
45                 wait(0);
46                 buf_index = 0;
47             }
48         }
49         else
50         {
51             arg[buf_index++] = buf;
52         }
53     }
54     exit(0);
55 }
```

测试结果:

```
make: "kernel/kernel"已是最新。
== Test xargs == xargs: OK (6.0s)
(Old xv6.out.xargs failure log removed)
```

完成实验一全部内容，在 xv6 目录下创建 time.txt 文件，写入实验耗时，在终端执行 ./grade-lab-util 对实验进行总体评分，结果如下：

```
make: "kernel/kernel"已是最新。
== Test sleep, no arguments == sleep, no arguments: OK (3.3s)
== Test sleep, returns == sleep, returns: OK (2.3s)
== Test sleep, makes syscall == sleep, makes syscall: OK (2.9s)
== Test pingpong == pingpong: OK (2.5s)
== Test primes == primes: OK (2.8s)
== Test find, in current directory == find, in current directory: OK (3.7s)
== Test find, recursive == find, recursive: OK (4.9s)
== Test xargs == xargs: OK (4.7s)
== Test time ==
time: OK
Score: 100/100
```

五、总结

操作系统的第一次实验，通过五个任务，对 xv6 的系统调用和一些基本概念有了更深入的了解。通过 sleep 的调用熟悉了 lab 的流程；通过 pingpong 进一步学习了管道的创建、进程间通信的实现过程；通过 primes 中递归调用实现了筛选素数的功能，是作业二的加深运用，父进程等待子进程结束有了更深刻的理解；通过 find 和 xargs 实现了查找程序和标准输入转为命令行参数的功能。本次实验让我对操作系统尤其是进程和管道的部分有了更深刻的理解。